EPAM L2 TECHNICAL ROUND

PRIORITY 4 SCENARIO-BASED INTERVIEW GUIDE

FULL SAMPLE ANSWERS • INCLUDING AI IN QUALITY ENGINEERING

Realistic interviewer questions followed by first-person answers you can practise aloud.

How this version is different

This edition keeps the scenario coverage but adds complete first-person model answers. Each answer explains what I would do, why I would do it, how I would communicate the decision, and how I would measure success. Adapt the wording to your real experience; do not claim examples or outcomes you have not personally delivered.

Interview technique
Start with your decision, then explain the steps and reasoning. Close with the risk, communication plan and measurable outcome. Aim for around 60–120 seconds for most answers; complex leadership scenarios may take longer.

B. Automation strategy and framework architecture

The sample answers use a senior SDET / Test Automation Lead voice. Practise the logic, not just the wording.

SCENARIO 01 — Migrate 2,000 Selenium tests to Playwright in one week

Interviewer: You have two engineers and 2,000 Selenium tests to migrate in one week. How would you plan it? What if the full migration is not achievable?

Sample interview answer:

"First, I would clarify what 'migrated' means. Is the expectation only to convert the scripts, or must all tests also pass, be code-reviewed, use stable test data, and run reliably in CI? Those are very different commitments.

I would then assess the existing suite. I would identify critical business journeys, test complexity, duplicated tests, dependencies, and current failure rates. I would select a representative sample—simple, medium, and complex tests—and migrate it as a proof of concept. That gives us real data about effort, framework gaps, and the level of rework required.

In parallel, I would establish the Playwright baseline: project structure, fixtures, page objects or other agreed abstractions, locator rules, test-data strategy, reporting, and CI execution. Once that baseline is stable, I would split work by feature area rather than have both engineers modify the same files. I would introduce code reviews and daily integration checkpoints to detect inconsistencies early.

I would prioritise smoke tests and high-risk customer journeys first, followed by the rest of the regression suite. I would track tests converted, passing, blocked, flaky, and integrated into CI. I would not report a converted test as complete if it does not run reliably.

If the initial throughput shows that all 2,000 cannot be completed safely, I would raise that early and present options: migrate the highest-risk scope by the deadline, add experienced resources, reduce scope by removing duplicates or low-value tests, or extend the timeline. I would recommend the option that protects release risk and gives stakeholders a clear, evidence-based forecast."

Likely follow-up questions: How would you calculate capacity? What would you do if the first proof of concept reveals major framework changes? What is your definition of done?

SCENARIO 02 — Tests pass locally but fail in CI

Interviewer: The Playwright suite passes on laptops but fails intermittently in Jenkins. How do you investigate?

Sample interview answer:

"I would first avoid assuming the application is defective or that the timeout is too short. I would classify the failures and collect evidence from CI: traces, screenshots, browser logs, test output, and timestamps. I would compare the local and CI environments, including browser versions, operating system, dependency lockfile, environment variables, test data, network access, CPU and memory limits, and parallel execution settings.

Next, I would reproduce the failure using the same browser and execution settings as CI. If the trace shows an element was not ready, I would investigate the actual condition the test should wait for instead of adding a fixed sleep. If the failure appears only in parallel, I would check for shared accounts, reused data, static variables, or shared browser contexts. If it is environment-specific, I would check service availability, configuration and permissions.

After fixing the root cause, I would rerun the affected test repeatedly and run the relevant suite in parallel. I would monitor whether the failure rate has reduced. I would use retries only as a diagnostic or controlled resilience mechanism, not as a substitute for fixing flakiness. Finally, I would document the cause and add a prevention check if the same class of issue could recur."

Likely follow-up questions: What evidence would you collect first? When are retries appropriate? How do you distinguish an environment failure from a product defect?

SCENARIO 03 — Parallel execution causes random failures

Interviewer: Tests pass sequentially but fail when executed in parallel. What are the likely causes and how would you fix them?

Sample interview answer:

"My first suspicion would be shared state or resource contention. I would check whether tests use the same user account, modify the same record, share static variables, reuse browser contexts, write to the same file, or depend on execution order. I would also check whether the application or a test dependency enforces rate limits.

I would use logs and timestamps to identify overlapping actions and reproduce the issue with a controlled number of workers. Then I would isolate test data, generate unique records where possible, use a separate browser context per test, and remove order dependencies. If a shared resource is unavoidable, I would manage access deliberately and document the reason rather than adding synchronisation everywhere.

I would rerun the suite repeatedly at the intended parallelism and compare the failure rate with sequential execution. Success means the tests remain independent and reliable under the expected load—not merely that one run passes."

Likely follow-up questions: Would you reduce the worker count? What is the difference between a race condition and shared test data?

SCENARIO 04 — A critical test is flaky before a release

Interviewer: Checkout fails intermittently in CI but passes manually. The release is tomorrow. What do you do?

Sample interview answer:

"I would treat this as a release risk until I had evidence to classify it. I would inspect the failure history and trace, identify the exact step that failed, and check whether the cause is timing, data, an environment dependency, or a genuine application issue. I would reproduce it under CI-like conditions and verify whether the test is exercising a critical customer journey.

If it is a test defect, I would fix the root cause and repeat the test enough times to establish confidence. If it reveals a product defect, I would raise it through the release process. If the cause remains unknown, I would communicate that uncertainty instead of labelling the failure as false. I would present the impact, evidence, mitigation options, and residual risk to the release owner. Any temporary quarantine would be documented, time-bound, and paired with an owner and follow-up date.

I would not simply increase timeouts or rerun until the test turns green, because that can hide a real problem. My recommendation would be based on customer impact and evidence, not pressure to make the dashboard green."

Likely follow-up questions: Would you ever quarantine a test? What information must be shared before a release decision?

SCENARIO 05 — The team asks for 100% automation

Interviewer: A manager wants every test case automated. How would you respond?

Sample interview answer:

"I would first ask what outcome the manager wants—faster regression, fewer escaped defects, shorter release cycles, or reduced manual effort. A 100% automation target can be misleading because not every test provides value when automated, and some activities—such as exploratory testing, usability assessment, and investigating unexpected behaviour—need human judgement.

I would classify the suite by risk, frequency, repeatability, business impact, and maintenance cost. I would prioritise stable, repeatable regression checks and critical workflows. I would automate business rules at the API or component layer when that gives faster and more reliable feedback, while keeping a smaller set of end-to-end UI tests for important user journeys.

I would propose measurable targets such as coverage of critical business risks, regression execution time, suite stability, maintenance effort, and escaped defects. That gives us a more meaningful view of progress than a raw automation percentage. If the business still wants a percentage target, I would agree on a defined scope and explain the trade-offs openly."

Likely follow-up questions: How would you decide what not to automate? How would you show ROI?

C. Delivery pressure and leadership

Use clear decisions, realistic forecasts, and transparent communication.

SCENARIO 06 — One engineer resigns halfway through a deadline-critical project

Interviewer: You have two engineers migrating a suite. After two days, one resigns. What do you do?

Sample interview answer:

"My first step would be to understand the delivery impact, not to blame the person who is leaving. I would review their work in progress, pending changes, blockers, and any knowledge that exists only with them. I would arrange a respectful handover if possible and make sure the current state is documented in the repository or team knowledge base.

Then I would recalculate capacity and identify the critical path. I would reassign work based on the remaining engineer's strengths and my own ability to contribute, while avoiding two people changing the same area without coordination. I would prioritise critical smoke and regression journeys and pause low-value work if needed.

I would tell the manager and stakeholders promptly that capacity has changed, explain the impact on scope and forecast, and offer options: protect critical scope by the deadline, add a qualified resource, or adjust the date. I would not pretend the original plan remains realistic, and I would avoid relying on unsustainable overtime as the recovery strategy. I would track progress daily and review whether documentation or cross-training needs to improve for future work."

Likely follow-up questions: How would you handle the remaining engineer's stress? What if management refuses extra time or resources?

SCENARIO 07 — Two team members conflict and one threatens to resign

Interviewer: Two engineers disagree repeatedly about ownership and technical decisions. One says they may resign. How do you handle it?

Sample interview answer:

"I would address the issue promptly, but I would not assume that one person is right before understanding the facts. I would speak with each person privately, listen to their perspective, and ask for specific examples. I would try to identify whether the root cause is unclear ownership, competing technical opinions, workload, communication style, or inappropriate behaviour.

Once I understand the issue, I would clarify responsibilities and how technical decisions should be made. If the disagreement is about design, I would define objective criteria—maintainability, reliability, delivery risk, and team standards—and facilitate a focused discussion. If it is a behaviour issue, I would set clear expectations for respectful collaboration. I would agree on concrete next steps and follow up rather than assuming one conversation resolves everything.

If there are allegations of bullying, harassment, or serious misconduct, I would use the appropriate HR process. If someone still chooses to resign, I would respect that decision, arrange a knowledge handover where possible, and protect delivery continuity. I would also reflect on whether unclear processes or earlier warning signs contributed to the conflict."

Likely follow-up questions: What if both people blame each other? What if the person is a top performer but behaves badly?

SCENARIO 08 — The deadline is fixed but the scope is unrealistic

Interviewer: Management says the date cannot move, but testing scope exceeds the team's capacity. What do you do?

Sample interview answer:

"I would first clarify which outcomes are genuinely mandatory for a safe release and which can be phased. I would assess risk by customer impact, likelihood, financial or regulatory consequences, and the availability of other controls. I would then create options—for example, complete the critical journeys and highest-risk regression by the date, add qualified capacity, reduce lower-value scope, or move the date.

I would explain the trade-offs in business terms. I would not say simply that the team cannot do it; I would show what can be completed, what remains untested, the risks of each option, and the mitigation available. If stakeholders choose to release with residual risk, that decision should be explicit and owned by the appropriate release authority.

After agreement, I would track the plan against measurable checkpoints and raise any change in forecast early. The goal is to give stakeholders control over the trade-off rather than surprise them at the deadline."

Likely follow-up questions: How do you rank critical tests? What if a senior leader asks you to mark testing complete anyway?

SCENARIO 09 — A production defect escapes despite high automation coverage

Interviewer: A serious defect reaches production even though automation coverage is high. How do you lead the retrospective?

Sample interview answer:

"First, I would support containment and help the team understand customer impact. Once the immediate issue is under control, I would lead a blameless but accountable analysis of how the defect escaped. I would trace the defect back through the requirement, test design, test data, environment, execution and release evidence.

I would ask whether the requirement was ambiguous, whether the right risk was covered, whether the test asserted the correct business outcome, whether the relevant data or configuration was represented, and whether the pipeline actually ran the test. High test count or line coverage does not guarantee that the important behaviour is validated.

I would agree on a specific corrective action, such as adding a missing boundary or negative test, improving requirement reviews, adding API-level validation, or strengthening release checks. Each action would have an owner and due date. I would then verify that the new control detects the defect class and review whether similar flows have the same gap. I would focus on improving the system rather than blaming an individual."

Likely follow-up questions: How would you measure whether the corrective action worked? What if the defect was caused by an ambiguous requirement?

D. API, debugging and CI/CD

Explain what you validate and why; avoid status-code-only or dashboard-only reasoning.

SCENARIO 10 — A third-party API is unavailable during testing

Interviewer: The payment provider is down. How can the team continue testing without hiding integration risk?

Sample interview answer:

"I would separate the testing objectives. We can continue validating our service's behaviour using a controlled mock, for example WireMock, while clearly identifying those tests as mocked integration or component tests. I would model expected responses and failure modes such as timeouts, declined payments, malformed responses, rate limits, and retry behaviour. I would also verify that our service handles failures safely and does not create duplicate transactions.

However, a mock cannot prove that the real provider integration works. I would keep a smaller set of real integration tests for when the provider is available, and I would monitor the dependency's status separately. I would also verify that the mock contract matches the agreed API contract, and update it when the provider's behaviour changes.

In the test report, I would distinguish tests that passed against the mock from tests that passed against the real service. That lets the team keep moving without overstating integration confidence."

Likely follow-up questions: How do Mockito and WireMock differ? What failure modes would you simulate?

SCENARIO 11 — An API returns 200 but the business operation failed

Interviewer: The response is HTTP 200, but the body says the payment was declined. How do you test it?

Sample interview answer:

"I would assert more than the HTTP status. HTTP 200 tells me that the server successfully handled the HTTP request, but it does not necessarily mean the business operation succeeded. I would validate the response schema, business status, error code or decline reason, and the expected resource state. If the contract says a declined payment must not create a completed order, I would verify that invariant as well.

I would also check that the test distinguishes expected business outcomes from unexpected technical errors. For example, a valid decline may be a successful API response from a contract perspective, while a malformed response or server exception may be a technical failure. The assertions should reflect the documented API contract and business rules rather than assuming every 2xx response means the business action succeeded."

Likely follow-up questions: Would a decline always be HTTP 4xx? How would you validate the response schema?

SCENARIO 12 — Pass an ID from one request to another

Interviewer: You create an order in one API request and need its ID in the next request. How would you implement it safely?

Sample interview answer:

"After the create request, I would assert that the response is successful and that the returned ID is present and valid. I would extract the ID and store it in a local variable or a scenario-scoped test context, then use that value in the next request—for example, to retrieve or update the same order.

I would avoid storing the ID in a static global variable because parallel tests could overwrite each other's values and cause intermittent failures. I would keep the data lifecycle within the test or scenario, and clean up the created order if the test environment supports it. If the next request fails, I would include the correlation ID and relevant response details in the report so the flow can be debugged."

Likely follow-up questions: How would you do this with REST Assured? Why is static state risky in parallel execution?

SCENARIO 13 — Repeated PUT or DELETE behaves unexpectedly

Interviewer: How do you test idempotency for PUT and DELETE?

Sample interview answer:

"I would start from the API contract and define the expected resource state after the first request and after repeated identical requests. For PUT, repeating the same update should leave the resource in the same intended state. For DELETE, repeating the operation should not recreate or further corrupt the resource. The response code for the second request might differ depending on the contract, so I would not require identical status codes unless that is explicitly specified.

I would verify the resulting state and any required side effects, such as audit records or emitted events. I would also test missing-resource behaviour and, where relevant, concurrent requests. The important assertion is that repeating the operation does not produce additional unintended effects."

Likely follow-up questions: How does POST differ from PUT? Is PATCH always idempotent?

SCENARIO 14 — Concurrent requests create duplicate payments

Interviewer: Two requests arrive almost simultaneously and create duplicate transactions. How would you investigate and test a fix?

Sample interview answer:

"I would reproduce the issue in a safe test environment using coordinated concurrent requests, unique correlation IDs, and the same business operation or idempotency key as defined by the contract. I would capture the responses and inspect persisted state to confirm whether two transactions were created.

I would work with the developers to inspect idempotency handling, transaction boundaries, locking, and database uniqueness constraints. The automation should assert the business invariant—for example, that only one payment is created for one idempotent operation—not merely that both requests return an expected status code.

After the fix, I would repeat the test under controlled concurrency and verify both response behaviour and final state. I would also check related failure cases, such as a client retry after a timeout, because the server may have processed the first request even if the client did not receive the response."

Likely follow-up questions: How can a timeout create duplicate requests? How would you make concurrency testing repeatable?

E. AI-enabled Quality Engineering

AI scenarios need the same engineering discipline plus explicit privacy, evaluation and human oversight.

SCENARIO 15 — AI generates tests with incorrect assertions

Interviewer: Your AI coding assistant generates tests from user stories. The tests compile and pass, but may not validate the business rule. What do you do?

Sample interview answer:

"I would treat AI-generated tests as a first draft, not as evidence that the requirement is covered. I would trace every important assertion back to an acceptance criterion and verify that the expected result is correct. I would review positive, negative, boundary, and state-transition cases, especially those with financial or customer impact.

I would run the tests against controlled data and review the test oracle—the logic that decides whether the result is correct. A test can pass consistently and still be wrong if it asserts the wrong outcome. I would require normal code review, and I would consider adding mutation testing or seeded defects to see whether the tests detect meaningful failures.

I would measure the value of AI by the quality of coverage, time saved after review, and defects detected—not by the number of tests generated. Any generated code would still need to meet the team's framework, security, and maintainability standards."

Likely follow-up questions: What is a test oracle? How would you prove that a test detects the intended defect?

SCENARIO 16 — AI generates hundreds of flaky UI tests

Interviewer: The AI-generated suite is large, slow and flaky. How would you improve it?

Sample interview answer:

"I would first assess whether the suite adds meaningful risk coverage or simply duplicates existing tests. I would group tests by business behaviour, remove duplicates, and identify assertions that belong at a lower test layer. Business rules that can be tested reliably through an API should not all be repeated through the UI; I would retain UI tests for critical user journeys and integration behaviour.

For flaky tests, I would inspect traces and failures, stabilise locators, isolate test data and browser contexts, remove fixed sleeps, and wait for observable application conditions. I would also review whether the AI-generated tests rely on implementation details or create unrealistic data dependencies.

I would compare the suite before and after changes using runtime, first-attempt pass rate, flakiness, maintenance effort, critical risk coverage, and defects detected. The goal is a smaller, trustworthy suite that gives fast feedback—not a high test count."

Likely follow-up questions: How would you prioritise tests to remove? What metrics define flakiness?

SCENARIO 17 — AI suggests brittle XPath or code that hides failures

Interviewer: An AI assistant suggests absolute XPath, hard waits, or broad exception handling. How do you govern it?

Sample interview answer:

"I would apply the same engineering standards to AI-generated code as to code written by a person. I would reject absolute or positional locators when a stable test ID or accessible locator is available, and I would replace fixed sleeps with condition-based waits. I would also reject broad exception handling that swallows failures, because it can make a broken product appear to pass.

To prevent recurrence, I would provide the assistant with repository-specific instructions and good examples, and I would reinforce standards through linting, pull-request reviews, templates, and automated checks where possible. I would look for patterns in rejected suggestions and improve the guidance. The final responsibility for correctness and maintainability remains with the engineer who reviews and merges the code."

Likely follow-up questions: Can repository rules guarantee correct AI output? What checks can be automated?

SCENARIO 18 — An engineer may paste confidential data into an AI tool

Interviewer: An engineer wants to paste production logs, tokens, or customer details into an external AI tool to debug a failure. What do you do?

Sample interview answer:

"I would stop the data from being shared until we confirm the tool is approved for that data classification. Production tokens, personal information, and confidential customer data should not be pasted into an unapproved external service. I would use an approved enterprise tool or sanitise the logs by removing secrets and identifiers, and use synthetic data when possible.

If a credential has already been exposed, I would follow the incident process and arrange rotation or revocation as appropriate. I would also explain the policy to the engineer and help them create a minimal, redacted reproduction so the technical issue can still be investigated. I would not treat convenience as a reason to bypass privacy and security controls."

Likely follow-up questions: What would you do if the secret had already been exposed? How would you sanitise logs?

SCENARIO 19 — AI-generated tests pass, but customers report defects

Interviewer: How do you evaluate whether AI is actually improving quality?

Sample interview answer:

"I would investigate the escaped defects and determine why the tests did not detect them. I would check whether the requirement was represented correctly, whether the test data covered the relevant conditions, whether assertions were meaningful, and whether the test suite exercised the affected path. I would also check whether the tests were actually executed in the release pipeline.

I would compare AI-assisted testing with a baseline using measures such as review and rework time, meaningful risk coverage, defects detected, escaped defects, maintenance effort, and feedback time. Where practical, I would use mutation testing or seeded defects to assess whether the suite detects known faults. I would also review generated tests with domain experts, because a plausible-looking test can encode the wrong business assumption.

If AI increases test volume but does not improve detection or makes maintenance worse, I would narrow its use to tasks where it provides measurable benefit."

Likely follow-up questions: What is mutation testing? How do you compare fairly against a baseline?

SCENARIO 20 — Testing an LLM or chatbot when answers vary

Interviewer: You are testing a chatbot, and exact text matching is unsuitable. How do you design reliable tests?

Sample interview answer:

"I would begin by defining what a good response means for the product. For example, does it answer the question correctly, stay grounded in approved information, protect private data, refuse disallowed requests, and provide relevant information? I would turn those expectations into a documented evaluation rubric with representative examples.

I would build a versioned dataset that includes normal use, edge cases, ambiguous prompts, adversarial attempts, and safety-sensitive cases. For objective requirements, I would use deterministic checks—for example, whether a required citation is present or a secret is exposed. For semantic quality, I would combine rubric-based evaluation with human-reviewed samples. If an LLM judge is used, I would calibrate it against human ratings and track disagreement rather than treating its score as absolute truth.

I would set thresholds for release, track model and prompt versions, and investigate regressions across the evaluation set. Because outputs can vary, I would evaluate patterns and distributions across cases rather than rely on a single exact response."

Likely follow-up questions: How do you evaluate hallucination? What are the limitations of an LLM judge?

SCENARIO 21 — AI service is unavailable on release day

Interviewer: Your test-generation workflow depends on an AI service, but it becomes unavailable. Does testing stop?

Sample interview answer:

"I would design the workflow so that validated regression execution does not depend on live AI availability. Once generated tests are reviewed and merged, they should be stored in source control and run through the normal CI pipeline like any other tests. AI can help create or analyse tests, but it should not be a runtime dependency for the release gate unless that dependency is explicitly designed and approved.

If the AI service is unavailable, the team should use the existing test suite, standard debugging tools, and manual analysis as a fallback. I would communicate which AI-assisted activities are delayed, but I would not stop running already validated tests. After recovery, I would assess whether the workflow needs better fallback procedures, caching, or clearer separation between test creation and test execution."

Likely follow-up questions: Would you allow an AI service to decide release readiness? What should be version-controlled?

F. More AI and leadership follow-ups

Use these to deepen your answers when the interviewer challenges your first response.

ifferent scores on repeated runs	“I would first test the evaluator's reliability rather than assume the applicat calibration set, define the rubric with examples, and measure agreement would pin model and prompt versions where possible, repeat a sample of Objective checks should remain deterministic where possible. I would not gate.”
ut review time rises	“I would measure the full delivery cycle: time to generate, review time, rev experience. I would compare similar tasks with a baseline and segment b than complex framework design. I would improve repository context and p tasks. If the net benefit remains negative, I would reduce or stop that use
partial refund cases	“I would start with domain-led risk analysis rather than asking the AI to inv duplicate submissions, timeouts, partial failures, state transitions, bounda constraints and acceptance criteria to the assistant, then review its output verify business invariants such as no duplicate refund and correct final ba
cts your recommendation	“I would ask the engineer to explain the technical reasoning and compare reliability, maintainability, delivery risk and operational cost. If the evidenc proof of concept. Once a decision is made, I would document the rationale I would distinguish a healthy technical challenge from repeated behaviour
s skipped tests and retries	“I would inspect what actually executed, what was skipped, first-attempt fa critical-path coverage. A green summary is not enough if the evidence is i where possible, explain the remaining uncertainty, and give the release o improve the dashboard so skipped tests and retry-masked failures are vis

G. How to practise these answers

Turn model answers into your own interview responses.

• Read the interviewer question, then answer aloud before reading the sample.

• Compare your answer with the model: did you clarify the goal, explain the decision, mention risks and describe how you would verify success?

• Replace generic wording with a real example from your experience, but do not invent metrics or results.

• Keep the opening direct. Avoid long background explanations before stating your decision.

• For AI scenarios, consistently mention appropriate data handling, human review, evaluation quality and fallback where relevant.

• Prepare for follow-ups by explaining the technical details behind each claim.

A useful closing sentence
“I would verify the result with evidence, communicate any remaining risk, and make sure the corrective action has an owner and a follow-up date.”

End of guide • EPAM Priority 4 — Scenario-Based Interview Guide with Full Sample Answers

AGENTIC AI EXPANSION

Supplement to the EPAM Priority 4 Scenario-Based Interview Guide

This addendum expands the original sample answers where an agentic AI workflow could realistically help. It is designed to give you an interview-ready way to propose AI agents without implying that agents can guarantee correct code or remove the need for engineering judgement.

Key distinction
An agentic system is more than several prompts. It has an orchestrator, bounded tools and permissions, task state, feedback loops, validation gates, failure handling and human approval. Agents may propose or perform actions, but code changes and release decisions still need controlled verification.

Where to introduce agentic AI

- **Scenario 01:** Selenium-to-Playwright migration — strongest fit for a multi-agent workflow.
- **Scenarios 02–04:** CI failure triage, flaky-test diagnosis and parallel failure clustering.
- **Scenario 05:** Test selection and risk-based coverage recommendations; avoid automating every test blindly.
- **Scenarios 10–14:** API contract analysis, mock generation, test-data setup and failure diagnosis, with deterministic assertions.
- **Scenarios 15–21:** AI-generated tests, LLM evaluation, privacy and fallback — these require explicit evaluation and governance.
- **Scenarios 08–09:** Agents can summarise risk and evidence, but cannot own release-risk acceptance or replace human accountability.

1. Scenario 01 — Agentic migration system for 2,000 tests

Use this expanded answer if you want to demonstrate AI innovation and delivery judgement.

Interviewer asks: How would you migrate 2,000 Selenium tests to Playwright in one week with two engineers?

Interviewer: Would you build an agentic system to accelerate the migration? How would it work and how would you ensure quality?

Sample interview answer:

"I would propose an agentic migration workflow to reduce repetitive conversion effort, but I would not assume it can safely migrate all 2,000 tests without measurement. First, I would define the target: a migrated test must compile, follow our framework standards, pass its assertions, be reviewed, and run in CI. I would also inspect the current framework, test dependencies, test-data setup and locator quality.

I would build an orchestrator that reads the test inventory, classifies tests by complexity and business criticality, assigns bounded tasks, records status, and retries or escalates failures within limits. An analysis agent would inspect each Selenium test and identify locators, waits, assertions, page-object dependencies and test data. A conversion agent would generate Playwright code using our approved templates, fixtures and locator rules. A review agent would check the result for banned patterns, missing assertions, hard waits, unsafe exception handling and consistency with the target architecture. A validation runner would execute the generated test in a controlled environment and capture traces and reports.

I would not allow the agents to merge changes directly into the main branch. They would create proposed changes in a separate branch or pull request, and engineers would review complex tests, business assertions and failures. The workflow would maintain traceability from each original test to its generated version, validation result and reviewer.

I would begin with a representative pilot covering simple, medium and complex tests. I would measure successful conversion rate, review effort, first-run pass rate, defect rate and time per validated test. That evidence would drive the forecast. If one week is not enough, I would propose a risk-based first delivery—critical smoke and business journeys first—and show the scope, staffing or date trade-offs. The agentic system accelerates repeatable work; it does not remove the need for realistic estimation or human accountability."

Important controls: Use least-privilege repository access; never expose secrets to the model; sandbox execution; cap retries; log agent actions; validate assertions against requirements; require pull-request review; keep a human-owned release decision.

A practical agent workflow

Component	Responsibility	Output / gate
Orchestrator	Creates batches, tracks status, handles retries and routes failures.	Traceable task state; bounded retries.
Analysis agent	Reads Selenium test and identifies selectors, waits, assertions, dependencies and complexity.	Migration plan per test.
Conversion agent	Generates Playwright code using approved templates and framework rules.	Draft code in isolated branch.
Review agent	Checks coding standards, locator quality, assertions and risky patterns.	Findings; not an automatic approval.
Execution validator	Runs tests, gathers traces, screenshots, logs and result status.	Pass/fail evidence and diagnostics.
Human engineer	Reviews semantics, business correctness, complex cases and merge decisions.	Approved pull request and release confidence.

Metrics to mention

Validated tests per day • first-run pass rate • percentage requiring manual correction • review time • flaky-test rate • critical-risk coverage • traceability to original tests.

2. Where else agentic AI is applicable

Bring it in when it adds controlled leverage—not just to make the answer sound modern.

Scenarios 02–04 — CI failures, flaky tests and parallel execution

Interviewer: Could agents help diagnose intermittent failures across a large automation suite?

Sample interview answer:

"Yes. I would use a bounded triage workflow. A failure-analysis agent could group failures by stack trace, locator, environment, test data and time window. A diagnostics agent could inspect approved logs, Playwright traces and screenshots to propose likely causes, such as shared data, a timing assumption, browser-version drift or a dependency outage. A remediation agent could suggest a patch or a focused reproduction, but it would not automatically suppress the test or merge code.

The workflow would attach evidence to each hypothesis and ask an engineer to confirm the root cause. After a fix, CI would rerun the affected test repeatedly and under the intended parallelism. I would track time to diagnose, recurrence rate and first-attempt pass rate. I would not let an agent increase timeouts, add retries, or quarantine tests without an explicit policy and review."

Important controls: Use read-only log access by default; redact secrets and personal data; retain evidence; prohibit silent test suppression; require review for code changes.

Scenario 05 — Risk-based test selection

Interviewer: How could AI agents help if management asks for 100% automation?

Sample interview answer:

"I would not use agents to blindly generate tests for every existing case. I would use an analysis agent to cluster duplicates, map tests to requirements and impacted components, and recommend priorities based on business criticality, change history and defect history. A separate review step would check that the recommendation does not omit regulatory or high-risk paths. The team would then agree which cases to automate, retain as manual exploratory tests, combine, or retire. I would report risk coverage and maintenance cost rather than only automation percentage."

Important controls: Recommendations must be explainable and reviewed; coverage mapping and risk rules should be visible to stakeholders.

Scenarios 10–14 — API testing and debugging

Interviewer: Would you use agents to create API tests or diagnose API failures?

Sample interview answer:

"I would use an API-analysis agent to read approved API specifications and suggest positive, negative, boundary and contract tests. It could draft request/response examples, mock mappings and test-data setup. A validation step would compare these drafts with the published schema and business rules, while deterministic API tests would verify status, schema, business outcomes, idempotency and persisted state. For failures, an agent could correlate request IDs, logs and responses to suggest likely causes, but the test assertions and final diagnosis would still be verified by an engineer."

Important controls: Use synthetic test data; protect credentials; validate against versioned API contracts; never accept generated assertions solely because they pass.

Scenarios 08–09 — Scope trade-offs and escaped defects

Interviewer: Can an agent decide what testing to drop or whether a release is safe?

Sample interview answer:

> “An agent can help assemble evidence—for example, tests affected by a code change, untested high-risk areas, recent defect history and execution results—and present possible scope options. However, I would not delegate release-risk acceptance to an agent. The accountable product or release owner, supported by engineering and QA, must decide whether residual risk is acceptable. I would ensure that assumptions, evidence, mitigations and the decision owner are recorded.”

Important controls: Agents can recommend and summarise; accountable humans make risk acceptance and release decisions.

3. Strengthen the AI-specific answers

These additions connect the existing AI scenarios to practical agentic-system design.

Scenarios 15–17 — Generate, review and improve tests

Interviewer: How would you design an agentic workflow for AI-generated tests?

Sample interview answer:

"I would separate test generation from test approval. First, an analysis agent would extract acceptance criteria and propose a coverage matrix. A test-generation agent would draft test cases and code. A review agent would map assertions back to requirements and flag missing negative, boundary and state-transition cases. The execution system would run the tests in a sandbox and report results. A human reviewer would validate whether the assertions reflect the real business rule before the tests could be merged.

I would use feedback from reviewed failures to improve templates and instructions, but I would not allow the agents to mark their own work as correct without independent checks. I would measure assertion correctness, review effort, useful defects detected, and maintenance cost—not just test count."

Important controls: Use independent validation; do not let the generator be the sole judge of its own output; version prompts, models and generated artifacts.

Scenarios 18–19 — Privacy and escaped defects

Interviewer: How would you make an agentic testing workflow safe for company code and customer data?

Sample interview answer:

"I would first define which models and tools are approved for the repository and data classification. Agents would receive only the minimum necessary context, with secrets and personal data redacted. Tool permissions would be restricted: for example, a diagnostic agent may read sanitised logs, while a code-generation agent may create a branch but not merge it or change production systems. Every tool call and generated change would be logged.

I would also evaluate the workflow itself. I would sample generated tests, review escaped defects, compare against a baseline, and test whether the agents can be tricked into unsafe actions or incorrect conclusions. If the workflow increases test volume but not meaningful defect detection, I would narrow or redesign it."

Important controls: Least privilege, approved tools, sanitised inputs, audit trail, human approval, and a way to disable the workflow safely.

Scenario 20 — Testing an LLM product

Interviewer: Would you build an agent to evaluate an LLM product?

Sample interview answer:

"Potentially, but I would separate test generation, evaluation and final release decisions. An agent could propose adversarial prompts and edge cases from a documented risk taxonomy. The evaluator would score responses against a versioned rubric for correctness, relevance, groundedness, safety and privacy. For objective properties I would use deterministic checks; for semantic quality I would calibrate automated scores against human-labelled examples. I would track model and prompt versions and analyse regressions across a representative dataset.

I would not let a single LLM judge be the only authority, because it can be inconsistent or share the same blind spots as the model being tested. A human-reviewed calibration set and clear release thresholds are essential."

Important controls: Version the datasets and rubrics; measure agreement with human ratings; test adversarial behaviour; keep human ownership of release gates.

Scenario 21 — AI service outage

Interviewer: How would you make an agentic workflow resilient if the AI service is unavailable?

Sample interview answer:

> “I would separate test generation from test execution. Validated tests would be committed to source control and run by standard CI without calling the AI service. If the AI endpoint is down, generation and diagnosis may be delayed, but the known-good regression suite should still run. I would define timeouts, bounded retries, error handling, service health monitoring and a manual fallback. I would not make a live AI agent a single point of failure for a release gate unless the system's purpose explicitly requires it and the risk is accepted.”

Important controls: Validated tests remain runnable without AI; clear fallback; bounded retries; no uncontrolled loops or autonomous production changes.

A concise interview summary

“I would use agents for repeatable analysis, code drafting, test selection and failure triage. I would keep permissions bounded, validate outputs independently, require human review for business assertions and code merges, and measure the benefit against a baseline. AI should accelerate the quality process, not replace engineering accountability.”